Field Hours

CSCE 222 Discrete Structures for Computing

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 50 minutes

Logic and a proof

A proposition is true or false. And, or, and not combine them. Implication P implies Q is false only when P is true and Q is false. The contrapositive, not Q implies not P, is the same statement. The converse, Q implies P, is a different statement. Most wrong proofs prove the converse and stop.

A proof is a sequence of sentences, each following from the previous or from a definition. Write the definition you are using. If you cannot name it, you are not proving yet.

Same implication, two directions.

Statement

If it is a square, it has four sides

Contrapositive: fewer than four sides, so not a square

Not the statement

Converse: four sides, so it is a square

A rectangle is the counterexample

One false implication

1. P: the number is 6. Q: the number is even. P implies Q is true for this P.

2. Q implies P is false: 4 is even and is not 6.

3. The contrapositive of P implies Q is 'odd, so not 6', which is true.

Problem 1. You proved that if a graph is a tree, it is connected. Have you proved that every connected graph is a tree?

Answer. No. That is the converse. A triangle is connected and is not a tree. The original implication can hold while the converse fails.

Problem 2. P implies Q is false. What are the truth values of P and Q?

Answer. P is true and Q is false. Every other combination makes the implication true.

Lecture 2 · 50 minutes

Sets, functions, relations

A set is a collection with no duplicates and no order. Intersection, union, and difference are the operations you

will see in specifications. A function assigns each element of the domain exactly one element of the codomain. A relation is any set of pairs. 'Equals' is a relation. 'Less than' is a relation. Not every relation is a function.

Injection means different inputs give different outputs. Surjection means every output is hit. Bijection means both, and it is why two sets have the same size when you can pair them. Infinite sets still use this definition.

A function and a relation that is not one.

Function

Each person has one birthday

Two people may share a day

Not a function

Each person has a set of friends

One person maps to many

Three students, two majors

1. Ada and Ben map to computer science. Cho maps to math. The map is a function: one major each.

2. It is not injective: Ada and Ben collide.

3. If the codomain is only those two majors, it is surjective. Add 'history' and nobody maps there, so it stops being surjective.

Problem 1. A roster lists the same student twice. As a set, how many elements is that?

Answer. One. Sets ignore duplicates. If the roster is a list, it has two entries, and that is a different type. Say which one you mean.

Problem 2. Why does a bijection between two finite sets prove they have the same number of elements?

Answer. Every element on the left pairs with a distinct element on the right, and nothing on the right is left over. The pairing is a count.

Lecture 3 · 50 minutes

Induction

Ordinary induction proves a claim for every integer n at least n0. Base case: check n0 by hand. Inductive step: assume the claim for k, and prove it for k + 1. The assumption is called the inductive hypothesis. You must use it. A step that proves k + 1 from scratch did not need induction, or it hid a gap.

Strong induction assumes the claim for every value up to k, not only k. Use it when the step looks back more than one, as merge sort does when it splits in half. The base still has to be checked.

The shape of an inductive proof.

1. Check the base

2. Assume k

3. Prove k + 1 using that assumption

4. Conclude every n past the base

The sum 1 + 2 + ... + n

1. Claim: the sum is n(n + 1) / 2. Base n = 1: 1 = 1 times 2 / 2.

2. Assume it for k. The sum to k + 1 is the sum to k, plus k + 1, which is k(k + 1) / 2 + (k + 1).

3. Factor (k + 1): (k + 1) (k / 2 + 1) = (k + 1)(k + 2) / 2. That is the claim at k + 1.

Problem 1. Your inductive step assumes the claim for every n, then concludes it for every n. What is wrong?

Answer. You assumed the theorem you were proving. The hypothesis is only for one k, or for values up to k. The step has to reach the next value from those.

Problem 2. A recurrence T(n) = T(n - 1) + n, with T(1) = 1. What does induction give?

Answer. T(n) = 1 + 2 + ... + n = n(n + 1) / 2. The base is T(1). The step adds n at the new size, which is the same sum as the example.

Lecture 4 · 50 minutes

Counting

If a choice has a options and then b options, and every pair is possible, there are a times b outcomes. If the choices are exclusive alternatives, you add. Mixing add and multiply is the usual mistake. Draw the decision.

A permutation of k items out of n is n! / (n - k)!. A combination ignores order, so divide by k! as well. Passwords care about order. Committees do not.

Order, or not.

Permutation

A code 1 then 2 differs from 2 then 1

P(n, k) = n! / (n - k)!

Combination

A committee {1, 2} is one set

C(n, k) = P(n, k) / k!

Choose 2 of 4

1. People A, B, C, D. Ordered pairs: 4 times 3 = 12.

2. Unordered pairs: AB, AC, AD, BC, BD, CD. Six.

3. 12 / 2! = 6. The division removes the two orders of each pair.

Problem 1. A password is 3 distinct digits, order matters. How many?

Answer. 10 times 9 times 8 = 720. Combinations would be wrong because 123 and 321 are different passwords.

Problem 2. You added the options instead of multiplying. When is addition the right rule?

Answer. When the cases cannot happen together. One topping from a list of 5 savory or a list of 4 sweet is 5 + 4, if you pick exactly one list. A savory and a sweet is 5 times 4.

Lecture 5 · 50 minutes

Graphs as mathematics

A graph here is a set of vertices and a set of edges, not yet a search algorithm. The degree of a vertex is the number of edges that touch it. The handshaking lemma says the sum of degrees is twice the number of edges, because each edge is counted at both ends. A corollary: the number of odd-degree vertices is even.

A path does not repeat vertices. A cycle does not, except the start. A tree is a connected graph with no cycle. It has one more vertex than edges. These sentences are the ones CSCE 411 will use without redefining them.

A tree on four vertices.

Four vertices

Three edges

Connected, no cycle

Sum of degrees is 6, which is twice the edges

Degrees 3, 1, 1, 1

1. Sum is 6, which is even, so the handshaking lemma allows it. There are three edges.

2. Three vertices have odd degree. Three is odd, and the corollary says the count of odd degrees is even. This degree list cannot be a simple graph.

3. Degrees 2, 2, 1, 1 sum to 6 and have two odd vertices. A path of length 3 has exactly that. It exists.

Problem 1. A connected graph has 10 vertices and 9 edges. What else do you know?

Answer. It is a tree. A connected graph with one fewer edge than vertices has no cycle. If it had 10 edges it would contain a cycle, or if it had 8 it would be disconnected.

Problem 2. Can a graph have degrees 1, 2, 3?

Answer. No. The odd degrees are 1 and 3, two of them, which is allowed by parity, but the sum is 6, so three edges. On three vertices a simple graph has at most three edges, a triangle, degrees 2, 2, 2. You cannot place a degree-3 vertex among three vertices without a loop or a double edge. Say simple graph when you claim impossibility.

Lecture 6 · 50 minutes

A small recurrence

A recurrence defines a function by smaller values. Unrolling writes the same equation until you see the pattern, then you prove the pattern by induction. T(n) = T(n / 2) + 1, with T(1) = 0, unrolls to one plus one plus one, log n times, if n is a power of two. So T(n) is log base 2 of n.

This is the cost of binary search. The graduate course will classify harder recurrences with the master theorem. You need this unrolling first, or the theorem is a slogan.

Unroll, then prove.

1. Write T at n

2. Substitute the smaller size

3. Count the substitutions until the base

4. Prove the closed form by induction

Binary search on 8 items

1. T(8) = T(4) + 1 = T(2) + 2 = T(1) + 3 = 3.

2. Log base 2 of 8 is 3. Each step halves the range and does one comparison.

3. Induction: base T(1) = 0 = log2(1). If T(k) = log2(k), then T(2k) = log2(k) + 1 = log2(2k).

Problem 1. T(n) = 2 T(n / 2) + n, T(1) = 0, n a power of two. Unroll one level and say what you see.

Answer. T(n) = 2 (2 T(n / 4) + n / 2) + n = 4 T(n / 4) + n + n. Each level adds n, and there are log n levels, so T(n) is n log n. The induction writes that claim and checks the step.

Problem 2. Why is the base case part of the closed form, not a detail?

Answer. The unrolling stops at the base. A wrong base adds a wrong constant at every leaf. For binary search, T(1) = 0 means an empty range costs nothing. T(1) = 1 would add one extra comparison you did not do.